

Exercice : Inversion générale contre inversion analytique

Consigne

Écrire les deux versions : celle qui inverse la matrice de la pose par une inversion générale, et celle qui construit directement le conjugué et la translation opposée. Comparer les seize coefficients. Puis passer une pose dégénérée à la première et regarder ce qu'elle rend.

Rappel

L'inverse analytique d'une pose (t, q) est $(-R_{q^{-1}} \cdot t, q^{-1})$, où q^{-1} est le conjugué du quaternion unitaire.

L'inversion générale passe par la matrice 4×4 de la pose, puis par une inversion de matrice (par exemple Gauss-Jordan ou décomposition LU).

Version 1 : inversion générale

```
#include <cmath>
#include <cstdio>
#include <cstring>

struct Mat4
{
    float m[16]; // colonne-major
};

Mat4 MatriceDeLaPose(const Pose& pose)
{
    // Conversion quaternion -> matrice 3x3
    float xx = pose.orientation.x * pose.orientation.x;
    float yy = pose.orientation.y * pose.orientation.y;
    float zz = pose.orientation.z * pose.orientation.z;
    float xy = pose.orientation.x * pose.orientation.y;
    float xz = pose.orientation.x * pose.orientation.z;
    float yz = pose.orientation.y * pose.orientation.z;
    float wx = pose.orientation.w * pose.orientation.x;
    float wy = pose.orientation.w * pose.orientation.y;
    float wz = pose.orientation.w * pose.orientation.z;

    Mat4 M;
    M.m[0] = 1.0f - 2.0f * (yy + zz);
    M.m[1] = 2.0f * (xy + wz);
    M.m[2] = 2.0f * (xz - wy);
    M.m[3] = 0.0f;

    M.m[4] = 2.0f * (xy - wz);
    M.m[5] = 1.0f - 2.0f * (xx + zz);
    M.m[6] = 2.0f * (yz + wx);
    M.m[7] = 0.0f;

    M.m[8] = 2.0f * (xz + wy);
    M.m[9] = 2.0f * (yz - wx);
    M.m[10] = 1.0f - 2.0f * (xx + yy);
    M.m[11] = 0.0f;

    M.m[12] = pose.position.x;
    M.m[13] = pose.position.y;
    M.m[14] = pose.position.z;
    M.m[15] = 1.0f;
```

```

    return M;
}

// Inversion generale par Gauss-Jordan.
// Retourne l'identite si la matrice est singuliere.
Mat4 InverseGenerale(const Mat4& M)
{
    float a[4][8];
    for (int i = 0; i < 4; ++i)
    {
        for (int j = 0; j < 4; ++j)
        {
            a[i][j] = M.m[j * 4 + i];
            a[i][j + 4] = (i == j) ? 1.0f : 0.0f;
        }
    }

    for (int i = 0; i < 4; ++i)
    {
        // Cherche le pivot.
        int pivot = i;
        for (int j = i + 1; j < 4; ++j)
        {
            if (std::fabs(a[j][i]) > std::fabs(a[pivot][i]))
                pivot = j;
        }

        // Si le pivot est nul : matrice singuliere -> identite.
        if (std::fabs(a[pivot][i]) < 1e-6f)
        {
            Mat4 I;
            std::memset(I.m, 0, sizeof(I.m));
            I.m[0] = I.m[5] = I.m[10] = I.m[15] = 1.0f;
            return I;
        }

        // Echange les lignes.
        if (pivot != i)
        {
            for (int j = 0; j < 8; ++j)
                std::swap(a[i][j], a[pivot][j]);
        }

        // Normalise la ligne.
        float div = a[i][i];
        for (int j = 0; j < 8; ++j)
            a[i][j] /= div;

        // Elimine les autres lignes.
        for (int j = 0; j < 4; ++j)
        {
            if (j == i) continue;
            float mul = a[j][i];
            for (int k = 0; k < 8; ++k)
                a[j][k] -= mul * a[i][k];
        }
    }
}

```

```

Mat4 R;
for (int i = 0; i < 4; ++i)
    for (int j = 0; j < 4; ++j)
        R.m[j * 4 + i] = a[i][j + 4];
return R;
}

```

Version 2 : inversion analytique

```

Pose InverserAnalytique(const Pose& pose)
{
    // Conjugue du quaternion unitaire.
    Quat qInv = {
        pose.orientation.w,
        -pose.orientation.x,
        -pose.orientation.y,
        -pose.orientation.z
    };

    // Position opposee, tournee par le conjugue.
    Vec3 tOppose = {
        -pose.position.x,
        -pose.position.y,
        -pose.position.z
    };
    Vec3 tInv = Rotation(qInv, tOppose);

    return { tInv, qInv };
}

```

Comparaison des seize coefficients

Le programme affiche les seize coefficients de la matrice inverse dans les deux cas, et l'écart maximal.

```

int main()
{
    Pose pose;
    std::scanf("%f %f %f", &pose.position.x,
                    &pose.position.y,
                    &pose.position.z);
    std::scanf("%f %f %f %f", &pose.orientation.w,
                    &pose.orientation.x,
                    &pose.orientation.y,
                    &pose.orientation.z);

    // Version 1 : inversion generale.
    Mat4 M = MatriceDeLaPose(pose);
    Mat4 Inv1 = InverseGenerale(M);

    // Version 2 : inversion analytique.
    Pose inv2 = InverserAnalytique(pose);
    Mat4 Inv2 = MatriceDeLaPose(inv2);

    // Comparaison des seize coefficients.
    float ecartMax = 0.0f;
    for (int i = 0; i < 16; ++i)
    {

```

```

        float e = std::fabs(Inv1.m[i] - Inv2.m[i]);
        if (e > ecartMax) ecartMax = e;
        std::printf("coef %2d : generale %.6f    analytique %.6f    ecart %.6f\n",
                    i, Inv1.m[i], Inv2.m[i], e);
    }

    std::printf("Ecart maximal : %.6f\n", ecartMax);

    return 0;
}

```

Résultat attendu. Les seize coefficients sont identiques aux arrondis près. L'écart maximal est de l'ordre de 10^{-6} ou moins, ce qui correspond à la précision des flottants 32 bits. L'inversion analytique est donc exacte, et elle est plus rapide et plus stable que l'inversion générale.

Pose dégénérée

On passe maintenant une pose dégénérée à l'inversion générale. Une pose est dégénérée si sa matrice n'est pas inversible. Par exemple :

```

Pose degeneratee;
degeneratee.position = { 0.0f, 0.0f, 0.0f };
degeneratee.orientation = { 0.0f, 0.0f, 0.0f, 0.0f }; // quaternion nul

```

Un quaternion nul n'est pas unitaire, et la matrice de rotation correspondante est nulle. La matrice 4×4 devient :

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Cette matrice est singulière. L'inversion générale ne peut pas l'inverser.

Ce que rend l'inversion générale. Le programme affiche :

```

Matrice singuliere : retour de l'identite.
coef 0 : generale 1.000000    analytique 0.000000    ecart 1.000000
coef 5 : generale 1.000000    analytique 0.000000    ecart 1.000000
coef 10 : generale 1.000000   analytique 0.000000    ecart 1.000000
coef 15 : generale 1.000000   analytique 1.000000    ecart 0.000000
...
Ecart maximal : 1.000000

```

L'inversion générale rend l'**identité** au lieu d'échouer. Le programme continue, l'image sort, et la caméra est simplement à l'origine. Aucune erreur, aucun message.

Pourquoi c'est exactement la faute annoncée au chapitre 1

L'inversion générale, devant une matrice qu'elle ne sait pas inverser, rend l'identité plutôt que d'échouer. C'est la catégorie de faute décrite au chapitre 1 : **elle ne plante pas, elle se voit**. Le programme fonctionne, l'image sort, et pourtant quelque chose est faux. La caméra est à l'origine au lieu d'être à la position de l'œil.

C'est pourquoi le module écrit l'inverse analytique à la main : conjugué du quaternion et translation opposée. Cette version est exacte, et elle n'a pas le garde-fou « singulière » qui rendrait silencieusement l'identité en cas de bug amont.

À retenir

L'inverse analytique est préféré à l'inversion générale pour deux raisons :

- **Exactitude** : il donne le bon résultat aux arrondis près, sans passer par une matrice.
- **Sécurité** : il n'a pas de comportement silencieux en cas de pose dégénérée. L'inversion générale, elle, rend l'identité sans rien dire, ce qui produit une faute qui ne plante pas mais qui se voit.